Triggers

Triggers

- Uma trigger é uma procedure que, associada a uma tabela (ou visão), é executada automaticamente quando é inserido, alterado ou removido um registro da tabela;

- Gatilhos: pois disparam códigos (armazenados no servidor) ou SQL's, sem a necessidade de uma chamada como nas Functions, mas sim a partir de uma situação satisfeita no servidor.

- Os comandos utilizados em um trigger são os mesmos das procedures, porém a execução da mesma não é feita pelo usuário e sim pelo SGBD na inserção, alteração ou remoção de registros;

Triggers

- Estes dão um pouco mais de independência aos DB os livrando do uso de aplicações ou usuários os manipulando.

- Automatização quando determinadas situações ocorrem no Banco de Dados.

EX: Um produto esgotado no estoque;

Um valor estimado ser alcançado no orçamento;

Validação de dados inseridos em uma tabela, etc.

- A Aplicação(SW) poderia ser a responsável por isto, mas teria que ser programada para executar de tempos em tempos estas verificações.

Vantagens da Triggers

- Nas triggers podemos observar as mesmas vantagens oferecidas pelas procedures:
  - facilidade de manutenção;
  - maior desempenho das aplicações;
  - redução do tráfego na rede;
  - melhora da segurança com o controle das regras de negócio diretamente no servidor.

Tipos de Triggers

- São classificados de acordo com sua execução que pode ser:
  - BEFORE: antes ou
  - AFTER: depois

- Antes e depois do que?
  - INSERT,
  - UPDATE ou
  - DELETE,

Possibilidades

- BEFORE INSERT - Trigger disparado **antes** de uma **inserção**
- BEFORE UPDATE - Trigger disparado **antes** de uma **atualização**
- BEFORE DELETE - Trigger disparado **antes** de uma **exclusão**
- AFTER INSERT - Trigger disparado **após** uma **inserção**
- AFTER UPDATE - Trigger disparado **após** uma **atualização**
- AFTER DELETE - Trigger disparado **após** uma **deleção**

- É possível combinar alguns modos desde que tenham a operação AFTER ou BEFORE em comum:
  - BEFORE INSERT OR UPDATE

Triggers

- Sintaxe:
  - CREATE TRIGGER *nome tipo_trigger eventos* ON *nome_tabela* FOR *tipo_execução*

  EXECUTE PROCEDURE *função(parâmetros);*

- Onde:
  - ***tipo_trigger:*** BEFORE ou AFTER;
  - ***eventos:*** INSERT, UPDATE ou DELETE;
  - ***tipo_execução:*** EACH STATEMENT(executado uma única vez) ou EACH ROW(executado para cada linha);
  - ***Função(argumentos):*** mostra qual função será executada (aqui que está representada toda a lógica da trigger).

Exemplo Prático

- Suponha que para cada nova conta criada no banco esta pode ser de pessoa física ou jurídica, e que seja necessário armazenar o total de cada tipo de conta em uma tabela chamada TOTAL_CONTAS.

- O trigger em questão será invocado a cada uma nova conta cadastrada na tabela CONTAS, e, de acordo com o tipo de conta , atualizará o devido registro a tabela TOTAL_CONTAS.

- Para a realização deste exemplo será necessário a criação de uma BD chamado “banco” com as tabelas “contas” e “total_contas”.

Exemplo Prático

```
--criação da tabela CONTAS
CREATE TABLE CONTAS(
numero int,
saldo float,
tipo char(1)
  );

--criação da tabela TOTAL_CONTAS
CREATE TABLE TOTAL_CONTAS(
tipo varchar(10),
quantidade int
  );

--inserção dos registros que gerenciam os tipos de contas, ambos zerados:
INSERT INTO TOTAL_CONTAS VALUES (‘FISICA’, 0);
INSERT INTO TOTAL_CONTAS VALUES (‘JURIDICA’, 0);
```

Exemplo Prático

--criação de uma função que incremente a tabela TOTAL_CONTAS de acordo com o tipo de conta:

```
CREATE FUNCTION incrementa_contas() RETURNS TRIGGER AS $$
BEGIN
   -- A variável NEW  representa no PostgreSQL os valores do registro que estará sendo     manipulado
    IF NEW.tipo = 'F' THEN
        UPDATE TOTAL_CONTAS SET QUANTIDADE = QUANTIDADE +1      WHERE TIPO = 'FISICA';
    ELSE
        UPDATE TOTAL_CONTAS SET QUANTIDADE = QUANTIDADE +1 WHERE TIPO = 'JURIDICA';
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

Exemplo Prático

```
--Criação da Trigger para que a função incrementaContas seja
  invocada a cada novo registro inserido:
-- Na tabela Contas

CREATE TRIGGER trigger_incrementa_contas AFTER INSERT ON
  CONTAS FOR EACH ROW EXECUTE PROCEDURE
  incrementa_contas();
```

Teste para o Exemplo

-- Insira contas de ambos os tipos. A cada INSERT, a tabela TOTAL_CONTAS será atualizada

```
INERT INTO CONTAS (numero, saldo, tipo) VALUES (1, 20.00, 'F');
INSERT INTO CONSTAS (numero, saldo, tipo) VALUES (2, 10.00, 'F');
INSERT INTO CONTAS (numero, saldo, tipo) VALUES (3, 23.00, 'F');
INSERT INTO CONTAS (numero, saldo, tipo) VALUES (4, 0.00, 'J');
INSERT INTO CONTAS (numero, saldo, tipo) VALUES (5, 1.99, 'J');
```

-- No final a Tabela Total contas deverá estar da seguinte maneira:

```
SELECT * FROM TOTAL_CONTAS;
```

FÍSICA = 3 e JURÍDICA = 2

Data Output | Explain | Messages | Histor

	tipo character varying(10)	quantidade integer
1	FISICA	3
2	JURIDICA	2

Alterando uma Trigger

- A melhor maneira de se alterar uma trigger é excluindo e a criando novamente.
  - Manter o script correto em um documento texto.

- A não ser que se deseja alterar o nome:

- Sintaxe:
  - ALTER TRIGGER *nome* ON *tabela* RENAME TO *novo_nome*;

Excluindo uma Trigger

- Sintaxe:
  - DROP TRIGGER *nome* ON *tabela opções*;

- Onde as opções podem ser:
  - CASCADE: todos os objetos diretamente dependentes serão excluídos junto com a trigger.
  - RESTRICT: somente a trigger será excluída.

- **Obs:** A exclusão da trigger não exclui a função na qual é chamada pela mesma.

Habilitando uma Trigger

- Habilitada automaticamente no ato de sua criação;
- Mas pode acontecer de uma tabela estar configurada para não executar uma trigger;
- Sintaxe:
  - ALTER TABLE *nome_tabela* ENABLE TRIGGER *nome_trigger*;
- Para Liberar todas as Triggers ligadas a tabela substitua o *nome_trigger* por *ALL.*

Desabilitando uma Trigger

- Uma vez habilitada a mesma pode ser Desabilitada;
- Sintaxe:
  - ALTER TABLE *nome_tabela* DISABLE TRIGGER *nome_trigger*;
- Para Desativar todas as Triggers ligadas a tabela substitua o *nome_trigger* por *ALL.*

Exemplo

- Desenvolva um Trigger que ao ser excluído um registro da tabela produto apareça uma mensagem que os dados do produto foram excluídos com sucesso

```
--1º Criar a function
CREATE OR REPLACE FUNCTION msg_produto_excluído()
RETURNS trigger AS $$
BEGIN
        RAISE NOTICE 'Produto Excluído com sucesso!';
    RETURN new;
END;
$$ LANGUAGE plpgsql;
```

Continuação

```
--2º Criar a trigger
CREATE TRIGGER t_prod_excluido
AFTER DELETE
ON produto
FOR EACH ROW
EXECUTE PROCEDURE msg_produto_excluído();

--3º Testando a trigger
insert into produto(descproduto, saldoestoque) values ('leite
  longa vida', 75);

DELETE FROM produto WHERE idproduto=8
```

Exercício

```sql
/* Criação de tabelas */

CREATE TABLE funcionarios
(
  codigo int primary key,
  nome varchar(100) NOT NULL,
  email varchar(200) NOT NULL,
  telefone varchar(14) NOT NULL,
  profissao varchar(150) NOT NULL,
  endereco varchar(100) NOT NULL,
  salario numeric(15,2)
);

CREATE TABLE funcionarios_auditoria
(
    log_id INT NOT NULL,
    data_criacao TEXT NOT NULL,
    operacao_realizada varchar
);


```

Exercício

```
22 /* funcao para auditoria */
23 CREATE OR REPLACE FUNCTION funcionario_log_function()
24 RETURNS trigger AS $$
25 BEGIN
26     IF (TG_OP = 'INSERT') THEN
27         raise notice 'log insert';
28         INSERT INTO funcionarios_auditoria (log_id, data_criacao, operacao_realizada)
29         VALUES (new.codigo, current_timestamp, ' Operação de inserção. A linha de código '
30             || NEW.codigo || 'foi inserido');
31         RETURN NEW;
32     ELSIF (TG_OP = 'UPDATE') THEN
33         raise notice 'log update';
34         INSERT INTO funcionarios_auditoria (log_id, data_criacao, operacao_realizada)
35         VALUES (NEW.codigo, current_timestamp, 'Operação de UPDATE. A linha de código '
36             || NEW.codigo || ' teve os valores atualizados ' || OLD || ' com ' || NEW.* || '.');
37         RETURN NEW;
38     ELSIF (TG_OP = 'DELETE') THEN
39         raise notice 'log delete';
40         INSERT INTO funcionarios_auditoria (log_id, data_criacao, operacao_realizada)
41         VALUES (OLD.codigo, current_timestamp, 'Operação DELETE. A linha de código '
42             || OLD.codigo || ' foi excluída ');
43         RETURN OLD;
44     END IF;
45     RETURN NULL;
46 END;
47 $$ language plpgsql;
48
```

Exercício

```
49  CREATE TRIGGER trigger_log_todas_as_operacoes
50  AFTER INSERT OR UPDATE OR DELETE ON funcionarios
51  FOR EACH ROW
52  EXECUTE PROCEDURE funcionario_log_function();
53
```